
Note su programmazione in C

Elementi di programmazione in C utili per il progetto

Per il progetto 5 occorre implementare l'assemblatore del computer Hack usando il linguaggio di programmazione C

Vediamo quindi alcuni elementi di programmazione in C che bisogna conoscere (ad esempio, la gestione dei file), ma questo non è da considerarsi un tutorial o corso di programmazione C.

Per gli esempi non useremo un IDE ma useremo un generico editor di testi, compilazione da linea di comando o con `Makefile`

Per replicare le esercitazioni esattamente come vengono fatte in aula, per chi usa Windows si rende necessario installare un emulatore di terminale come ad esempio Cygwin.

Altrimenti, per installare gcc fare riferimento a informazioni per il vostro sistema.

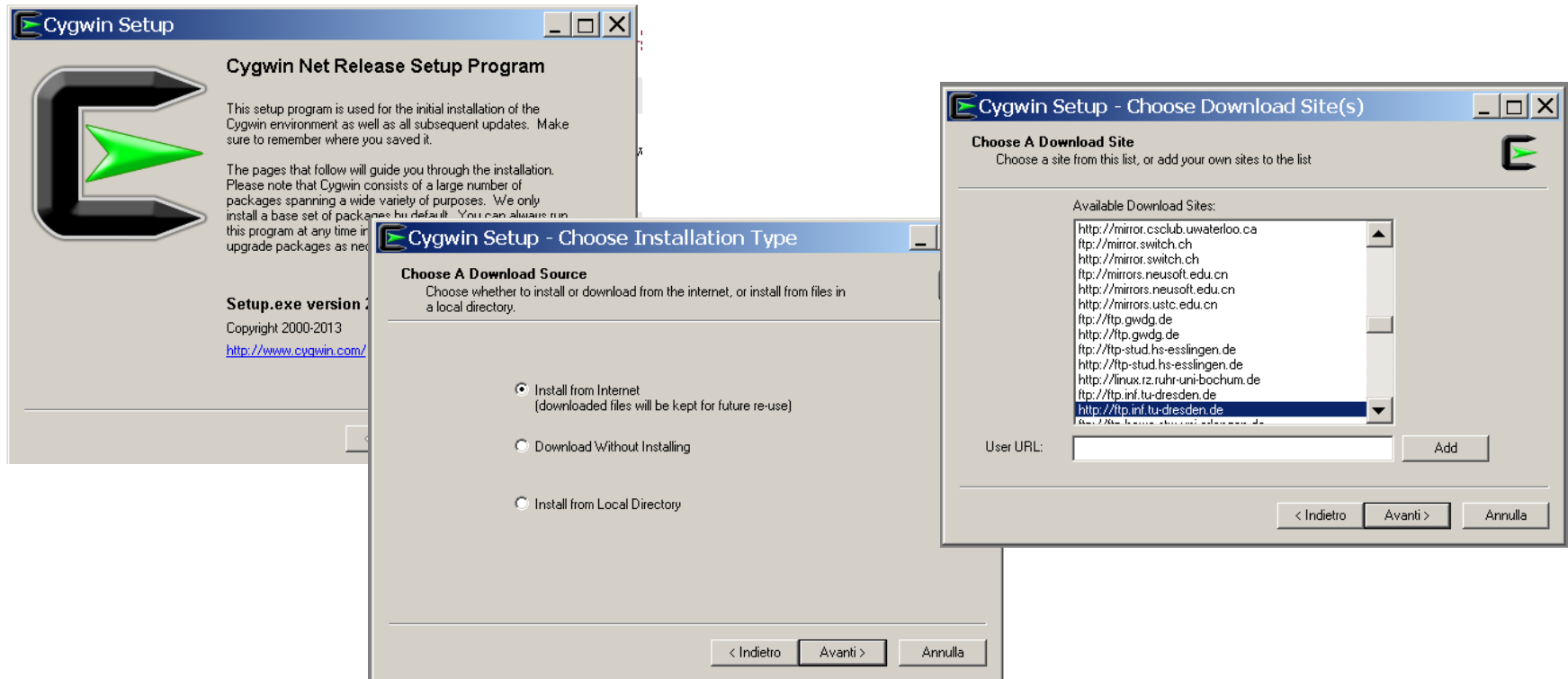
Installare Cygwin (per sistemi Windows)

Sul sito web www.cygwin.com

Consultare <http://cygwin.com/install.html>

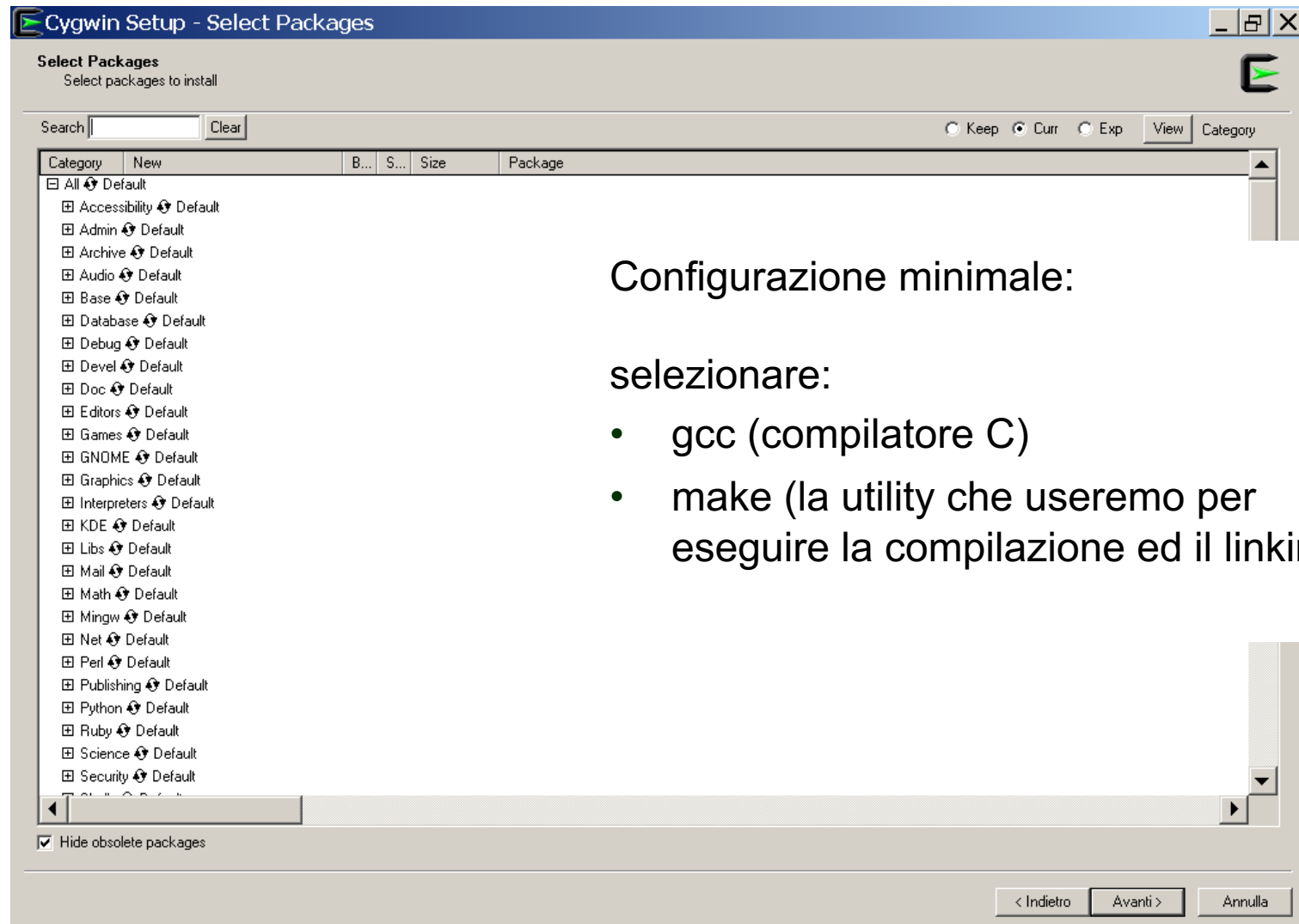
Scaricare ed eseguire `setup-x86.exe` (per sistemi a 32 bit) oppure `setup-x86_64.exe` (per sistemi a 64 bit).

Nota: x86 è una famiglia di ISA CISC



Installare Cygwin (per sistemi Windows)

Selezionare i pacchetti da installare dalle varie categorie



Configurazione minimale:

selezionare:

- gcc (compilatore C)
- make (la utility che useremo per eseguire la compilazione ed il linking)

Usare Cygwin – minimo indispensabile (per sistemi Windows)

Potete muovervi nella struttura delle directory con cd

- `cd /` va nella directory “root”
- `cd ..` va nella directory padre
- `cd nome1` va nella sottodirectory nome
- `cd nome1/nome2` va nella sottodirectory nome2 di nome1

Trovate i vostri dati nella directory `/cygdrive/X/` dove X è il drive

Potete richiamare notepad da riga di comando scrivendo notepad

Processo di compilazione in C

```
gcc helloworld.c
```

La compilazione C comprende le seguenti fasi:

Preprocessore:

- Trasforma i file sorgenti in base alle “*preprocessor directives*” specificate nel file

Compilatore:

- Traduce in assembly il programma in C
- Di default si genera direttamente il “*codice oggetto*” (*object file*), cioè un file già in linguaggio macchina (ancora con riferimenti ancora da istanziare, per esempio invocazione di funzioni di libreria: vedi linking). Quindi fa uso di un **assemblatore**.

Linking:

Collega vari *object files* e librerie (statiche) generando così un file eseguibile completo. Ne ripareremo più avanti nel corso.

Processo di compilazione in C

```
gcc helloworld.c
```

Opzioni interessanti:

```
gcc -E helloworld.c
```

Viene eseguito solo il **preprocessore**: non produce un object file .o

```
gcc -S helloworld.c
```

Viene eseguito il **compilatore** (senza **assemblatore**) ma non il **linker**: genera un file assembly simbolico

```
gcc -c helloworld.c
```

Viene eseguito il **compilatore** (incluso **assemblatore**) ma non il **linker**: genera un object file ma questo non è eseguibile

Preprocessore in C

Preprocessor directives che vediamo:

#include

Include un file esterno. Es:

```
#include <stdio.h>
```

#define MACRO, oppure **#define MACRO VALUE**

Definisce una macro, opzionalmente assegnandole un valore. Es:

```
#define PI 3.14
```

#ifdef MACRO .. #endif

Test: se una macro è definita allora include le direttive nel corpo, altrimenti vengono ignorate. Con **#ifndef** : testa se una macro non è definita. Es:

```
#ifndef test  
#define final  
#endif
```

Programmare usando file multipli

E' possibile separare in file distinti un programma C per avere una migliore organizzazione del proprio codice.

- Una volta scritto uno di questi file, se mancano gli altri, non è possibile eseguire il linking

Come permettere l'interazione fra i diversi file:

- Solitamente, ogni file `.c` viene affiancato da un relativo file `.h` che contiene le **dichiarazioni** (o prototipi) delle funzioni **definite** nel file `.c`
- Se si vogliono usare tali funzioni in un altro file, sarà sufficiente includere il file `.h` contenente le dichiarazioni. Il compilatore e linker si occuperanno del resto.

Se per praticità vogliamo evitare i file `.h`, dobbiamo comunque includere la dichiarazione delle funzioni nel file `.c` prima della loro definizione, altrimenti la compilazione potrebbe dare errori (a seconda del codice).

Esempio

```
// main.c
#include <stdio.h>
#include "somma.h"

int main() {
    printf("somma:%d\n", somma(5,3));
}
```

```
// somma.h
int somma(int i,int j);
```

```
// somma.c
#include "somma.h"
int somma(int i,int j){
    return i+j;
}
```

Compilazione dell'esempio

- Compilare (**senza** linkare) i file .c:

```
gcc -c main.c  
gcc -c somma.c
```

o

```
gcc -c *.c
```

Così si generano due object file: main.o e somma.o

- Effettuare il linking:

```
gcc -o main main.o somma.o
```

-o main specifica il nome dell'eseguibile. Default: a.out

Così si genera un eseguibile con nome: main

Per eseguire entrambe le fasi insieme:

```
gcc main.c  
gcc somma.c
```

o

```
gcc -o main *.c
```

Si esegue il file:

```
./main
```

“./” specifica alla shell che l'eseguibile è nella directory corrente

Makefile

Un modo per velocizzare (e razionalizzare) la compilazione in casi di programma scritto in più file è usare la utility make

Richiede la preparazione di un file chiamato Makefile. Esempio:

```
main: main.o somma.o
    gcc -o main main.o somma.o

main.o: main.c somma.h
    gcc -c main.c

somma.o: somma.c somma.h
    gcc -c somma.c

clean:
    rm *.o main
```

Makefile

Un modo per velocizzare (e razionalizzare) la compilazione in casi di programma scritto in più file è usare la utility make

Richiede la preparazione di un file chiamato Makefile. Esempio:

Target iniziale,
considerato
eseguendo: make

```
main: main.o somma.o  
    gcc -o main main.o somma.o
```

dipendenze con altri target

altro target

```
main.o: main.c somma.h  
    gcc -c main.c
```

comando da eseguire
per questo target

altro target

```
somma.o: somma.c somma.h  
    gcc -c somma.c
```

altro target di solito
aggiunto per ripulire
la cartella, eseguito
con: make clean

```
clean:  
    rm *.o main
```

I tab (indentazione)
come mostrati
nell'esempio sono
necessari

oppure `rm -f` per
sopprimere conferme

Considerazioni su make

Dopo la prima compilazione, ricompila solo le parti che sono state modificate:

- Scorre la catena delle dipendenze, e
- se trova un target che dipende da file modificati dopo l'ultima modifica del target, allora esegue il comando associato

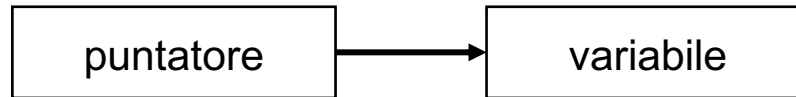
Tipi primitivi in C

Il linguaggio C mette a disposizione vari tipi primitivi che si differenziano per spazio occupato in memoria e valori che si possono rappresentare

Tipo	Size (byte)	Valore minimo	Valore Massimo
signed char	1	-128	+127
unsigned char	1	0	255
short int	2	-32768	+32767
unsigned short int	2	0	65535
long int	4	-2^{31}	$+2^{31}-1$
unsigned long int	4	0	$+2^{32}-1$
float	4	$-3.2 \times 10^{\pm 38}$	$+3.2 \times 10^{\pm 38}$
double	8	$-1.7 \times 10^{\pm 308}$	$+1.7 \times 10^{\pm 308}$
void	0	non definito	non definito

Puntatori in C

Un puntatore è un **tipo di dato**: una variabile di tipo puntatore contiene l'**indirizzo** in memoria di un'altra variabile, cioè un numero che indica in quale cella di memoria 'comincia' la variabile puntata.



In queste slide ogni riquadro raffigura **una o più** locazioni di memoria

Si possono avere puntatori a qualsiasi tipo di variabile.

La dichiarazione di un puntatore include il tipo dell'oggetto puntato. Es: un puntatore a interi punta alla locazione dove un intero è memorizzato.

Questo serve al compilatore, per sapere che tipo di dato troverà quando seguirà un puntatore e, quindi, la dimensione di quello che dovrà leggere (visto che un dato può occupare più locazioni, a seconda del tipo).

Per dichiarare un puntatore p ad una variabile di tipo **tipo**:

```
tipo *p;
```

```
int *p;
```

```
char *p;
```

```
int *p[2];
```

```
...
```

Operatori & e *

L'operatore & fornisce l'indirizzo di una variabile, perciò `p=&c` scrive nella variabile `p` l'indirizzo della variabile `c`, ovvero:

`tipo c, *p;` dichiaro una var `c` di tipo *tipo* (char, int, etc.) ed un
 puntatore `p` a *tipo*
`p = &c;` assegno a `p` l'indirizzo di `c`

L'operatore `*` viene detto operatore di indirezione (o *dereference*).

Quando una variabile **di tipo puntatore** è preceduta da `*`, si accede al dato puntato, ovvero: `*p` indica la variabile puntata dal puntatore `p`.

```
int c, *p;      p   c
p = &c;         p  →  c    // assegna a p l'indirizzo di c
c = 5;         p  → 5 c
int d = *p;     5 d    // equivale a: int d = c;

// *p e d ora valgono 5
```

Esempio

```
int x=1, y=2, z[10];          /* 2 int ed un array di int */
int *p;                       /* puntatore a int */

p = 4;                        /* ERRORE */
*p = 4;                       /* ERRORE : pointer non inizializzato */

p = &x;                       /* p punta a x */
y = *p;                       /* y ora vale 1. nota indirezione */
*p = 0;                       /* x ora vale 0. nota indirezione */
p = &z[0];                    /* p ora punta a z[0] */
```

Nota: in queste slide **non** ci occupiamo di studiare cosa sia scritto in stack vs heap, o in altri segmenti di memoria.

Stringhe in C

In C la stringa vera e propria **non** esiste.

Si rappresenta una stringa con un array (o vettore) di $n+1$ **char** (quindi char salvati in locazioni di memoria adiacenti) in cui in ultima posizione è assegnato il carattere nullo `'\0'`, che rappresenta il delimitatore finale, indicando che la stringa è finita. Es: una stringa "string":

's'	't'	'r'	'i'	'n'	'g'	'\0'
0	1	2	3	4	5	6

Una stringa (array di caratteri) viene scritta con doppi apici. Inizializzazione:

```
char s[] = "string";  
char s[7] = "string";           // almeno 7 !!  
char s[] = {'s','t','r','i','n','g','\0'};
```

L'allocazione di memoria è implicita.

Puntatori a char

Spesso, per stringhe useremo per comodità un puntatore a char.

```
char *s;
```

Con questa dichiarazione non allochiamo memoria!

```
char *s = "stringa";    //ora s+1 punta 't'
```

Qui "stringa" è un *literal string* ed in genere non è modificabile (è scritto in un segmento di memoria speciale) ed eseguire per esempio `s[1]='a'`; ritorna in genere errore (il comportamento è indefinito).

In questi casi usiamo invece array, che possiamo modificare a piacimento.

Libreria <string.h>

Questa libreria offre alcune funzionalità di manipolazione delle «stringhe» (l'header equivalente in C++ è <cstring>):

const esplicita il fatto che non viene modificata

```
#include <string.h>
```

```
char *strcpy(char *s1, const char *s2);  
//copia s2 nella stringa s1, incluso '\0'
```

```
char *strcat(char *dest, const char *src);  
//concatena la stringa src alla stringa dest
```

```
int strcmp(const char *s1, const char *s2);  
//confronta la stringa s1 con s2, ritornando 0 se uguali
```

```
... vedi string.h
```

Nota: strcpy non controlla la dimensione di dest. Il controllo va fatto prima, per evitare overflow. Come alternativa, possiamo usare invece:

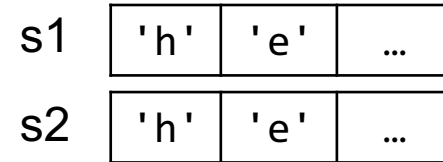
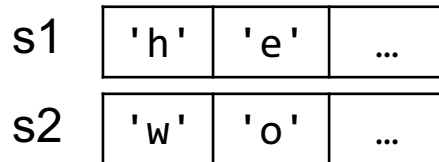
```
char *strncpy(char *dest, const char *src, size_t n); //con n=sizeof(dest)
```

alias per unsigned int. vedi sizeof() più avanti

Esempi

```
char s1[] = "hello";  
char s2[] = "world";
```

```
strcpy(s2,s1);
```

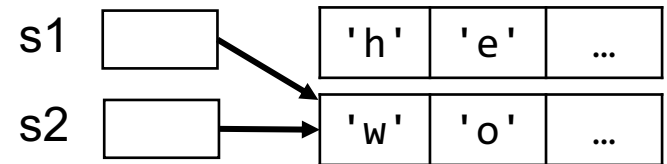
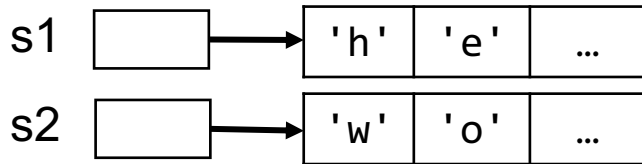


```
s1 = s2;           // errore: non possiamo assegnare array: usiamo strcpy  
  
s1 = "hello world"; // errore: come sopra  
  
strcpy(s1,"hello world"); // errore: s1 non ha lunghezza sufficiente  
  
char s3[];         // errore: inizializzare o specificare dimensione  
  
char s4[] = "hello world";  
strcpy(s4,"world"); // s4="world", non "world world": scrive '\0'
```

Esempi

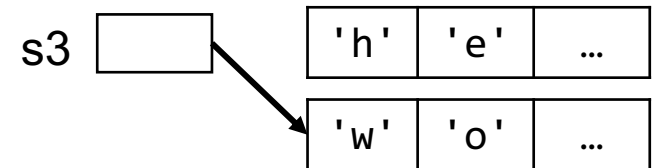
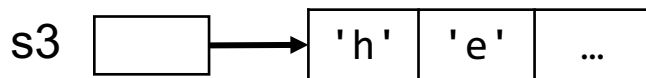
```
char *s1 = "hello";  
char *s1 = "world";
```

```
s1 = s2;
```



```
char *s3 = "hello";
```

```
char *s3 = "world";
```



```
char *s4;  
strcpy(s4, "hello"); // errore: memoria non allocata (s4 non inizial.)  
// vedi malloc() nelle prossime slide
```



Esempio: rimuovere spazi da una stringa

```
void delSpaces(char *string)
{
    int pos = 0;

    for (int i=0; string[i] != '\0'; i++)
    {
        if (string[i] != ' ')
        {
            string[pos] = string[i];
            pos++;
        }
    }

    string[pos] = '\0';
}
```

```
void delSpaces2(char* string,
                char* trimmed)
{
    while (*string != '\0')
    {
        if(*string != ' ')
        {
            *trimmed = *string;
            trimmed++;
        }
        string++;
    }
    *trimmed = '\0';
}
/* questa versione non modifica string */
```

Per i test usati, si potevano utilizzare funzioni offerte da librerie, come `isspace` (`ctype.h`), che confronta un `char` con caratteri di spazio, tabulazione, a capo.

Nota: questo codice poteva essere scritto in maniera più sintetica.

Nota: pass by pointer

Tecnicamente, in C le funzioni non ricevono argomenti "per riferimento" ma sempre "per valore". L'ostacolo è aggirato comunque con l'approccio "pass-by pointer» (possibile anche in C++): è il valore del puntatore (l'indirizzo) ad essere passato (via indirizzione implicita).

Ma a differenza di C++, non possiamo scrivere: `myfunction(int& i){}`

Tuttavia l'approccio è lo stesso (con qualche differenza.. es: potremmo passare un puntatore non inizializzato).

```
int main(){
    int i = 0;
    int *p = &i;

    myfunction(p); // or myfunction(&i);
    //...
}
void myfunction(int *q){
    //...
}
```

Output formattato

```
int printf(char *format, arg1, arg2, arg3, ...);
```

format è la stringa da stampare, la quale può contenere elementi (chiamati *format specifiers*) come quelli elencati sotto. La stringa stampata avrà questi specifiers sostituiti con gli argomenti arg1, arg2, ... passati, in ordine:

%d	stampa un intero
%10d	stampa un intero usando minimo 10 caratteri (padding a sx)
%-10d	stampa un intero usando minimo 10 caratteri (padding a dx)
%f	stampa un float
%lf	stampa un double
%10f	stampa un float usando minimo 10 caratteri (padding a sx)
%-10f	stampa un float usando minimo 10 caratteri (padding a dx)
%10.5f	stampa un float usando minimo di 10 caratteri (5 decimali)
%s	stampa tutti i caratteri di una stringa
%10s	stampa almeno 10 caratteri di una stringa (padding a sx)
%c	stampa un singolo carattere

```
char s[5] = "somma"; int a=3; int b=4;  
printf("La %s di %d e %d è %d\n", s, a, b, a+b);
```

La somma di 3 e 4 è 7

Input formattato

```
int scanf(char *format, *arg1, *arg2, *arg3, ...);
```

Viene letto l'input (da tastiera) e copiato nelle variabili agli indirizzi *arg1, *arg2,... come specificato da *format specifiers* nel format:

- %c viene letto un singolo carattere e copiato singolarmente nel primo byte indicato dal puntatore
- %10c vengono letti 10 caratteri e copiati nei primi 10 byte indicati dal puntatore
- %s viene letta la stringa e scritta interamente a partire dal puntatore passato, e come ultimo viene aggiunto un carattere '\0'
- %d viene letto un intero
- %f viene letto un float
- %lf viene letto un double

```
int a, b;  
printf("Scrivi due interi:\n");  
scanf("%d %d", &a, &b);  
printf("Letti: %d e %d\n", a, b);
```

passiamo indirizzi

Operatore sizeof()

L'operatore `sizeof` prende in input o una variabile o un identificatore di tipo, e restituisce la dimensione in byte. Esempio con un tipo:

```
printf("dimensione del float: %lu byte\n", sizeof(float) );
```

```
dimensione del float: 4 byte
```

Il dato può anche essere di un tipo definito dall'utente, non solo di un tipo di dato primitivo (predefinito).

Come visto qualche slide prima discutendo `strncpy`, `sizeof()` ritorna un risultato di tipo `size_t`, che è un alias per il tipo unsigned opportuno (dipendentemente dal sistema). Per stamparlo, possiamo usare `unsigned long` (format specifier: `%lu`) per portabilità.

Funzioni malloc e free

malloc() alloca dinamicamente la memoria (chiede al S.O. di riservare un'area di memoria) e restituisce un puntatore a void che punta all'inizio di quest'area di memoria allocata.

Se non è disponibile sufficiente memoria restituisce NULL, ad indicare che l'allocazione non è stata effettuata.

Per esempio, possiamo allocare memoria per una stringa:

```
char *s = (char *) malloc(10*sizeof(char));
```

Nota: malloc ritorna un puntatore void*. Il cast non è sempre necessario in C (a differenza di C++), ma fatto in queste slide per chiarezza. Fare attenzione a non oscurare potenziali errori sui tipi facendo cast espliciti che non sono necessari.

Funzioni malloc e free

E' possibile testarne il risultato:

```
int *p;
p = (int*) malloc( 100*sizeof(int) );           /* alloca 100 interi */

if(p==NULL) {
    printf("errore: allocazione impossibile");
    exit(1);
} else {
    /* .... uso la memoria */
    free(p);  /* rilascia la memoria allocata */
}
```

```
char *s4 = malloc( 50*sizeof(char) );
strcpy(s4,"hello");
```

Cast di tipo

La conversione di tipo **type-casting** in qualche caso viene effettuata implicitamente dal compilatore C, ma può anche essere forzata dal programmatore mediante un operatore unario detto **cast**:

(nome_nuovo_tipo) espressione

dove il *nome_nuovo_tipo* è un tipo di dato primitivo o definito dall'utente, ed *espressione* è una variabile / costante / espressione complessa.

```
int i=5, j=2;  
double f;  
f = i/j;  
printf("%d",f);
```

2

```
int i=5, j=2;  
double f;  
f = (double)i / (double)j;  
printf("%.2f",f);
```

2.50

Strutture in C

Le strutture in C sono record composti da più campi: `struct nome{...}`

Basandoci su una struttura, possiamo definire tipi di dato. Per esempio un tipo per rappresentare un punto geografico:

```
typedef struct luogo {          /* typedef definisce un tipo di
    char nome[30];              dato: in un questo caso
    int      x;                  una struttura */
    int      y;
    int      z;
};
```

Una volta definito il tipo di dato, possiamo dichiarare una variabile di quel tipo, premettendo però la keyword `struct`:

```
struct luogo bologna;
```

Uso della direttiva typedef

Possiamo dichiarare un alias a piacimento, per es. CITTA

```
typedef struct luogo {  
    char nome[30];  
    int    x;  
    int    y;  
    int    z;  
} CITTA;           // alias per il tipo
```

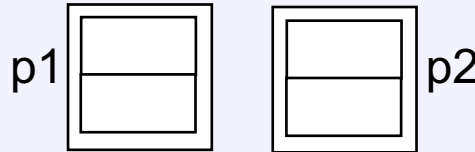
Una volta definito un alias lo possiamo usare nelle dichiarazioni di variabili, come fosse un tipo predefinito (senza ripetere struct):

```
CITTA bologna;
```

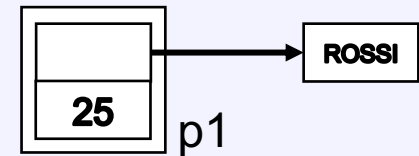
L'alias può essere uguale al nome (qui: 'luogo'). Sono due namespace separati.

Esempio: attenzione ai puntatori in strutture

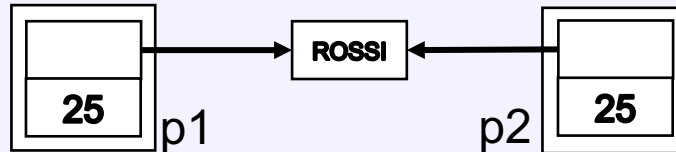
```
typedef struct {  
    char  *cognome;  
    int    eta;  
} persona;  
persona p1, p2;
```



```
// per p1.eta è già allocata memoria  
p1.cognome = (char*) malloc( 50 * sizeof(char));  
strcpy( p1.cognome, "ROSSI" );  
p1.eta = 25;
```



```
/* copio p1 */  
p2 = p1;
```



```
strcpy( p1.cognome, "NERI" ); // il cognome di entrambi cambia
```

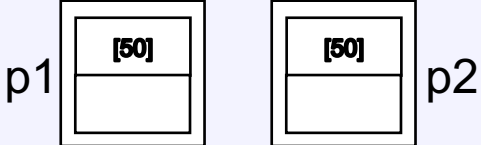
```
// invece con p1.cognome = "NERI" solo p1 sarebbe aggiornato
```

Esempio: attenzione ai puntatori in strutture (cont.)

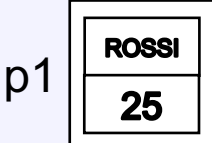
```
/* A differenza di prima, definiamo la struct con un array già  
   allocato per il cognome */
```

```
typedef struct {  
    char  cognome[50];  
    int   eta;  
}  persona;
```

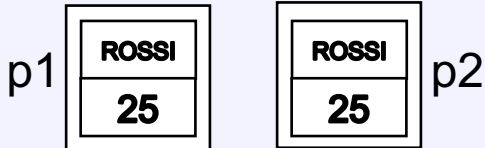
```
persona p1, p2;
```



```
strcpy( p1.cognome , "ROSSI" );  
p1.eta = 25;
```



```
p2 = p1;
```



```
strcpy( p1.cognome, "NERI" ); //stampare per verificare
```

Puntatori a strutture

E' possibile utilizzare puntatori che puntano a dati di tipo strutturato come le struct. La logica è la stessa.

```
/* ...struct persona come prima... */  
persona *p;  
persona p1;  
p = &p1;
```

In C esiste anche l'operatore $\rightarrow$ che permette l'accesso ad un campo della struttura puntata (infatti qui p è un puntatore a persona)

```
p -> eta = 25           // equivale a (*p).eta = 25
```

L'allocazione dinamica per strutture funziona come per altri tipi di variabile. Nel caso di campi di tipo puntatore, non puntano a memoria allocata.

```
persona *p2 = malloc( sizeof(persona) );
```

Esempio: semplice implementazione di uno Stack

Uno stack (tipo Stack) è un puntatore a una struttura chiamata `stackNode` di tipo `stackNode_t` (in particolare, il nodo in cima alla pila). Nell'invocare le varie funzioni, viene semplicemente passato sempre solo questo puntatore (con ovvi vantaggi).

```
//stack.h

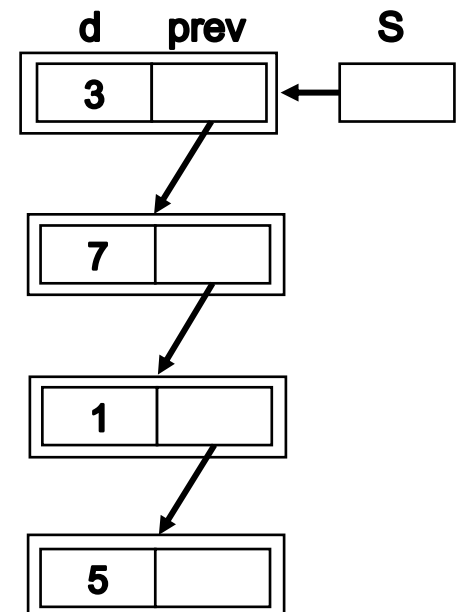
typedef struct stackNode {
    int d;
    struct stackNode *prev;
} stackNode_t; // alias del tipo

/* alias: tipo Stack e' un punt. a stackNode_t */
typedef stackNode_t *Stack;

void init(Stack *S);

void push(Stack *S, int d);

int pop(Stack *S);
```



Un esempio: semplice implementazione di uno Stack (cont.)

```
//stack.c

#include <stdio.h>
#include <stdlib.h>
#include "stack.h"

void init(Stack *S){
    *S=NULL;
}

void push(Stack *S, int d)
{
    Stack s = malloc(
        sizeof(stackNode_t));

    /* sia prev che *S sono puntatori */
    s->prev = *S;
    s->datum = d;
    *S = s;
}
```

```
int pop(Stack *S)
{
    int d;
    Stack old; // serve per free()

    if (*S != NULL)
    {
        d = (*S)->datum;
        old = *S;
        *S = (*S)->prev;
        free(old);
        return d;
    } else {
        printf("stack empty\n");
        exit(1);
    }
}
```

Input/output su file in C

Per poter accedere ad un file in lettura e/o scrittura, bisogna usare un puntatore a file

- Serve dichiarare delle variabili di tipo FILE*
- FILE è dichiarato nella libreria <stdio.h>

Per associare ad una variabile di tipo FILE* il proprio file si usa la funzione fopen

```
FILE *fin;  
fin = fopen("input.txt", "r");  
  
FILE *fout;  
fout = fopen("output.txt", "w");
```

I parametri "r" e "w" indicano apertura in lettura e scrittura rispettivamente ("w" crea il file se non esiste)

Comandi fprintf e fscanf

I comandi corrispondenti a printf e scanf, ma per scrivere/leggere file, sono fprintf e fscanf:

```
int a,b;  
fscanf(fin,"%d %d",&a, &b);  
fprintf(fout,"somma: %d\n",somma(a,b));
```

Specificando un pattern adeguato, può essere usata per leggere da file riga per riga (cf. documentazione).

La funzione feof restituisce true se si è tentato di leggere oltre la fine del file (EOF = 'end of file'):

```
if (feof(fin)){ ... }
```

Quando un file non viene più usato deve essere chiuso:

```
fclose(fin);      fclose(fout);
```

Lettura di una linea da un file di input

Una funzione apposita per leggere un'intera riga (cioè una sequenza di caratteri fino al carattere di fine linea, usualmente `\n`) da un file è `fgets`:

```
FILE *fp;  
char str[60];  
fp = fopen("file.txt" , "r");  
fgets(str, 60, fp);
```

Restituisce `NULL` se il file è terminato (se il primo carattere letto è `EOF`).

Legge fino a `n-1` caratteri, o fino a `'\n'` o `EOF`. Di solito si usa nel codice una costante, per esempio `MAX_LENGTH`, da passare come argomento.

La stringa letta (`str`) include il carattere nullo `'\0'`.

Restituisce un puntatore.

La funzione corrispettiva per scrivere una riga su file è `fputs`.

Esempio: copiare un file riga per riga

```
// parametri: nomi dei file
int copy(char source[], char target[]) {. // o copy(char *source, char *target)

    char buf[60]; // o quanto serve
    FILE *in, *out;

    in=fopen(source, "r");
    if( in==NULL ) {
        printf("Errore apertura di %s\n", source); exit(1);
    }
    out=fopen(target, "w");
    if( out==NULL ) {
        printf("Errore apertura di %s\n", target); exit(1);
    }

    while(fgets(buf, 60 ,in)!=NULL) {
        printf("ho letto la riga: %s", buf);           //newline è già nella riga letta
        fputs(buf, out);                               //scrive
        if( feof(in) )                                 //interrompe il ciclo se ha finito
            break;
    }
    fclose(in);
    fclose(out);
    return 0;
}

// nota: per riposizionarci di nuovo all'inizio di file in: rewind(in);
```

Parametri da linea di comando

E' possibile leggere i parametri della riga di comando. Per farlo è necessario dichiarare il main come:

```
int main(int argc, char **argv){}
```

- argc contiene il numero di argomenti, incluso il nome del programma;
- argv e' un array di stringhe, una per argomento, incluso il nome del programma (in argv[0])

Quindi per una funzione con n argomenti, $argc = n+1$.

Alcuni esercizi di pratica

- Scrivere un programma che prende da linea di comando il nome di un file e ne crea uno di estensione diversa ma con lo stesso nome. Vedi:
`sscanf(filename, "%s.%s", &nomefile, &estensione);`
- Scrivere un programma che prende da linea di comando il nome di 2 file di testo e per ogni riga letta dal primo scrive una riga sul secondo contenente la lunghezza di ciascuna parola escludendo spazi, tabulazioni, etc (il secondo viene creato se non esiste).
- Scrivere un programma che prende un parametro da command line e:
 - crea una lista di caratteri contenente i caratteri del parametro
 - elimina dalla lista il carattere in una posizione scelta dall'utente tramite input da tastiera
 - stampa a video la lista risultante